

Universidad de Costa Rica

Escuela de Ciencias de la Computación e Informática

CI-0119 P.I. de Arquitectura y Ensamblador

Proyecto Integrador de Arquitectura y Ensamblador

Segundo Avance

Número de equipo: 2

Profesor: Willian Baltodano Cubillo

Integrantes:

- Melissa Garita Chacón C23186

-Andrés Ramírez Solís C36467

- Jimena Bejarano C31074

-Stephanie Monge C35035

I. Proceso de diseño de máquina de estados

Objetivo

El sistema debe encender/apagar una luz con aplausos detectados en el sensor 1 y un ventilador con aplausos detectados en el sensor 2.

El control se realiza seleccionando un canal activo (canal 1 = luz, canal 2 = ventilador).

Las entradas y salidas se simulan con el Monitor Serial de Arduino IDE.

Estados definidos

- ESTADO_INICIAL: luz y ventilador apagados.
- ESTADO_LUZ_ENCENDIDA: sólo la luz encendida.
- ESTADO_VENT_ENCENDIDO: solo el ventilador encendido.
- ESTADO_AMBOS_ENCENDIDOS: luz y ventilador encendidos.

Eventos (entradas)

- aplauso: alterna Encendido/Apagado en el dispositivo del canal activo.
- canal1: selecciona directamente el canal de la luz.
- canal2: selecciona directamente el canal del ventilador.
- b: cambia de canal (simula un botón físico).
- estado: imprime el estado actual.

Acciones (salidas)

- `Serial.println("LUZ encendida.")`
- `Serial.println("LUZ apagada.")`
- `Serial.println("VENTILADOR encendido.")`
- `Serial.println("VENTILADOR apagado.")`
- `Serial.println("Estado actual: ...")`

Tabla de transiciones

Estado actual	Entrada	Acción	Estado siguiente
INICIAL	aplauso canal1	LUZ encendida	LUZ_ENCENDIDA
LUZ_ENCENDIDA	aplauso canal1	LUZ apagada	INICIAL
INICIAL	aplauso canal2	VENT encendido	VENT_ENCENDIDO
VENT_ENCENDIDO	aplauso canal2	VENT apagado	INICIAL
LUZ_ENCENDIDA	aplauso canal2	VENT encendido	AMBOS_ENCENDIDOS
VENT_ENCENDIDO	aplauso canal1	LUZ encendida	AMBOS_ENCENDIDOS
AMBOS_ENCENDIDOS	aplauso canal1	LUZ apagada	VENT_ENCENDIDO
AMBOS_ENCENDIDOS	aplauso canal2	VENT apagado	LUZ_ENCENDIDO

II. Implementación en Arduino IDE (C++)

El código se implementa en un archivo proyecto_et2.ino.

Está organizado en funciones que cumplen cada responsabilidad:

- ElegirCanal(): permite seleccionar el canal manualmente (**canal1** o **canal2**).
- CambiarCanal(): cambia entre canal 1 (LUZ) y canal 2 (VENTILADOR).
- Luces(): enciende/apaga la luz simulando un aplauso.
- Ventilador(): enciende/apaga el ventilador simulando un aplauso.
- ImprimirEstado(): muestra en Serial el estado actual del sistema.
- EstadoL() y EstadoV(): devuelven **true/false** según si la luz o el ventilador están encendidos.

III. Simulación (Pruebas)

Para validar el correcto funcionamiento del programa se realizaron pruebas utilizando el **Monitor Serial** del IDE de Arduino a 9600 baudios. En esta etapa no se utilizaron sensores ni actuadores físicos, por lo que las entradas fueron comandos escritos en el Monitor Serial y las salidas se observaron directamente en los mensajes impresos por el sistema.

Caso 1: Inicio del programa

Al iniciar la ejecución, el sistema muestra las instrucciones y el estado inicial:

--- Proyecto Aplausos ---

Canal inicial: LUZ

Opciones

canal1 -> seleccionar canal de luz

canal2 -> seleccionar canal de ventilador

b -> cambiar de canal (simula botón)

aplauso -> enciende o apaga

estado -> imprime estado actual

----- ESTADO ACTUAL -----

Canal activo: Luz

Luz: Apagada

Ventilador: Apagado

Caso 2: Encender la luz con un aplauso

Entrada:

Aplauso

Salida:

Luz encendida

Caso 3: Consultar estado actual

Entrada:

estado

Salida:

----- ESTADO ACTUAL -----

Canal activo: Luz

Luz: Encendida

Ventilador: Apagado

Caso 4: Cambiar al canal del ventilador

Entrada:

b

Salida:

Canal cambiado a: Ventilador

Caso 5: Encender el ventilador con un aplauso

Entrada:

aplauso

Salida:

Ventilador encendido

Caso 6: Consultar estado nuevamente

Entrada:

estado

Salida:

----- ESTADO ACTUAL -----

Canal activo: Ventilador

Luz: Encendida

Ventilador: Encendido

Caso 7: Apagar la luz después de seleccionarla

Entrada:

canal1

aplauso

Salida:

Canal seleccionado: Luz

Luz apagada

• Cómo se implementó la lógica de la máquina de estados en el Arduino IDE • Cómo se simuló el diseño en alto nivel de la aplicación en el Arduino IDE.